

LAB PRACTICUM: OpenMP Multi-Core Scaling in Python

Hands-Off Benchmark Practicum • 60 Minutes • Autonomous Execution

Student Name: Shynbay Daniyal **Student ID:** 230103158

Hardware / CPU: *AMD64 Family 25 Model 80 Stepping 0, AuthenticAMD, 12 Logical Cores*
Date & Section: 01.10.2026 (06-P)

🔗 LAB PROTOCOL & AUTONOMOUS RULES OF ENGAGEMENT

1. Autonomous Session: You are expected to read, run, optimize, and document all findings independently.
2. Zero Support for Dependency Issues: If you encounter an error, verify that you installed packages inside a clean virtual environment. Compilers are NOT needed for this lab.
3. Equal Hardware Standard: Keep your laptop plugged into wall power and close background browsers, video streams, or games to prevent CPU thermal throttling and frequency capping.
4. Whiteboard Leaderboard: When you finish Task 1 and Task 3, record your CPU model and best speedup factor on the lab whiteboard.

1. Background & Environment Setup (10 Minutes)

Standard CPython executes bytecode under the Global Interpreter Lock (GIL), preventing multiple CPU threads from executing simultaneously in Python space. In this lab, we bypass the GIL using Numba's JIT compiler with its native multithreaded OpenMP/TBB work-stealing engine. Because Numba compiles numeric Python loops directly to machine code and releases the GIL inside parallel loops (prange), we achieve bare-metal native CPU scaling across both macOS (Apple Silicon M-series) and Windows/Linux x86_64 machines with zero local C++ compiler dependencies.

Execute the following in your terminal / command prompt:

```
pip install numpy numba matplotlib
```

Instructor – sufyan.mustafa@sdu.edu.kz

2. Challenge 1: The Core Speedometer & Amdahl's Law

Objective: Benchmark an embarrassingly parallel Monte Carlo simulation to approximate π across varying thread counts. You will measure execution latency, quantify parallel speedup, and observe where Amdahl's Law caps performance.

```

import time
import numpy as np
import numba
from numba import njit, prange

print(f"Hardware Threads Detected: {numba.config.NUMBA_NUM_THREADS}")

@njit(parallel=True)
def monte_carlo_pi(n_samples):
    inside_circle = 0
    for i in prange(n_samples):
        x = np.random.uniform(0.0, 1.0)
        y = np.random.uniform(0.0, 1.0)
        if x * x + y * y <= 1.0:
            inside_circle += 1
    return (4.0 * inside_circle) / n_samples

# JIT Warmup (excludes JIT compilation overhead from benchmarks) _ =
monte_carlo_pi(10_000)

SAMPLES = 120_000_000
thread_counts = [1, 2, 4, 8, numba.config.NUMBA_NUM_THREADS]
# Filter out duplicates and invalid counts
thread_counts = sorted(list(set([t for t in thread_counts if t <=
numba.config.NUMBA_NUM_THREADS])))

print(f"{'Threads':<10} | {'Time (s)':<12} | {'Speedup':<10} | {'Efficiency (%)':<15}")
print("-" * 55)

t1_baseline = None
for t in thread_counts:
    numba.set_num_threads(t)
    start = time.perf_counter()
    pi_est = monte_carlo_pi(SAMPLES)
    elapsed = time.perf_counter() - start

    if t == 1:
        t1_baseline = elapsed
        speedup = 1.0
        efficiency = 100.0
    else:

```

Instructor – sufyan.mustafa@sdu.edu.kz

```

speedup = t1_baseline / elapsed
efficiency = (speedup / t) * 100.0

print(f"{'t':<10} | {'elapsed':<12.4f} | {'speedup':<10.2f}x |
{'efficiency':<15.1f}")

```

Deliverables & Technical Questions for Challenge 1:

- 📋 A. Record your baseline T_1 (1 thread) and your maximum multi-thread runtime T_{max} in Table 1 below.
- 📋 B. Why is the warm-up call `_ = monte_carlo_pi(10_000)` strictly required before recording benchmark timings?
- 📋 C. Did your parallel efficiency stay at 100% when scaling from 1 to max threads? What architectural factors (e.g. memory bus contention, hyperthreading/SMT core vs physical core limits) explain the degradation?
- 📋 D. Why does `inside_circle += 1` inside prange not corrupt the count? What implicit OpenMP reduction construct does Numba generate here?

```
Run challenge1 x
C:\Users\Daniyal Shynbay\AppData\Local\Programs\Python\Python313\python.exe "C:\Users\Daniyal Shynbay\Desktop\SDU 7 Semester\Parallel Comput
Hardware Threads Detected: 12
Threads | Time (s) | Speedup | Efficiency (%)
-----|-----|-----|-----
1 | 1.0753 | 1.00 | x | 100.0
2 | 0.5431 | 1.98 | x | 99.0
4 | 0.3026 | 3.55 | x | 88.8
8 | 0.2944 | 3.65 | x | 45.7
12 | 0.2423 | 4.44 | x | 37.0
Process finished with exit code 0
```

A. Recorded Timings & Speedup Metrics

Based on your execution results with **12 hardware threads** detected:

- **Baseline Single-Thread Runtime (T_1):** 1.0753seconds
- **Maximum Multi-Thread Runtime (T_{max} at 12 threads):** 0.2423seconds
- **Maximum Speedup Factor (S_{max}):** 4.44x
- **Maximum Parallel Efficiency (E_{max}):** 37.0%

B. Requirement of the JIT Warm-Up Call

- Executing `_ = monte_carlo_pi(10_000)` triggers Numba's Just-In-Time (JIT) compilation pipeline before benchmarking begins.
- During the first call, Numba inspects types, generates LLVM Intermediate Representation (IR), compiles native machine code binaries, and allocates the underlying OpenMP/TBB thread pool.
- Omitting the warm-up call would pollute the benchmark time with compiler latency rather than

measuring raw hardware performance.

C. Efficiency Degradation Analysis

Parallel efficiency dropped significantly from **100.0%** (1 thread) to **88.8%** (4 threads), **45.7%** (8 threads), and **37.0%** (12 threads) due to specific hardware limits:

1. **Physical vs. Logical Core Threshold (SMT / Hyper-Threading):** CPU possesses **6 physical cores** and **12 logical threads**. Scaling up to 4 threads yields near-linear speedup (3.55x) because each thread occupies a dedicated physical core. Scaling beyond 6 threads forces SMT pairs to share FPUs, ALUs, and L1/L2 caches, causing scaling to plateau.
2. **OpenMP Synchronization & Barrier Overhead:** Spawning, dispatching, and joining 12 logical threads incurs runtime management overhead at the implicit reduction barrier.
3. **Power & Frequency Constraints:** Sustained 100% utilization across all 12 logical processors triggers power/thermal constraints, slightly reducing single-core boost clock frequencies.

D. Reduction Mechanics and Thread Safety

- Writing `inside_circle += 1` inside a standard multi-threaded loop normally causes a data race (lost updates).
- In Numba's `@njit(parallel=True)`, the compiler analyzes variables inside `prange` and identifies accumulation operations as associative reductions.
- Numba automatically synthesizes an implicit OpenMP reduction construct:

```
#pragma omp parallel for reduction(+:inside_circle)
```
- Each thread maintains a private, thread-local accumulator on its local stack during execution. Upon loop completion, partial counts are safely merged via a lock-free binary reduction tree.

Instructor – sufyan.mustafa@sdu.edu.kz

3. Challenge 2: Load Imbalance & Dynamic Scheduling

Objective: Compute a high-resolution 2500×2500 Mandelbrot fractal. Unlike Monte Carlo, points in the complex plane require radically different amounts of compute: points inside the set iterate all the way to `max_iter`, while outer points escape in 2-3 iterations. You will investigate how non-uniform workloads impact thread utilization.

```

import time
import numpy as np
import matplotlib.pyplot as plt
from numba import njit, prange

@njit(parallel=True)
def render_mandelbrot_rows(h, w, max_iter):
    img = np.zeros((h, w), dtype=np.int32)
    # Row-parallel loop
    for r in prange(h):
        cy = -1.2 + (r / h) * 2.4
        for c in range(w):
            cx = -2.0 + (c / w) * 2.5
            z_real, z_imag = 0.0, 0.0
            it = 0
            while (z_real * z_real + z_imag * z_imag <= 4.0) and (it < max_iter):
                next_real = z_real * z_real - z_imag * z_imag + cx
                z_imag = 2.0 * z_real * z_imag + cy
                z_real = next_real
                it += 1
            img[r, c] = it
    return img

@njit(parallel=True)
def render_mandelbrot_cols(h, w, max_iter):
    img = np.zeros((h, w), dtype=np.int32)
    # Column-parallel loop (swapped iteration axis)
    for c in prange(w):
        cx = -2.0 + (c / w) * 2.5
        for r in range(h):
            cy = -1.2 + (r / h) * 2.4
            z_real, z_imag = 0.0, 0.0
            it = 0
            while (z_real * z_real + z_imag * z_imag <= 4.0) and (it < max_iter):
                next_real = z_real * z_real - z_imag * z_imag + cx
                z_imag = 2.0 * z_real * z_imag + cy
                z_real = next_real
                it += 1

```

```

img[r, c] = it
return img

# Warmup JIT
_ = render_mandelbrot_rows(100, 100, 50)
_ = render_mandelbrot_cols(100, 100, 50)

H, W, MAX_IT = 2500, 2500, 1000

t0 = time.perf_counter()
grid_rows = render_mandelbrot_rows(H, W, MAX_IT)
t_rows = time.perf_counter() - t0

t1 = time.perf_counter()
grid_cols = render_mandelbrot_cols(H, W, MAX_IT)
t_cols = time.perf_counter() - t1

print(f"Row-Parallel Render Time: {t_rows:.3f} s")
print(f"Column-Parallel Render Time: {t_cols:.3f} s")

# Save visual artifact
plt.figure(figsize=(8, 8))
plt.imshow(grid_rows, cmap='magma', extent=[-2.0, 0.5, -1.2, 1.2]) plt.title(f"Mandelbrot
{H}x{W} (Render: {t_rows:.2f}s)")
plt.axis('off')
plt.savefig('mandelbrot_output.png', dpi=300, bbox_inches='tight') print("Saved image:
mandelbrot_output.png")

```

Deliverables & Technical Questions for Challenge 2:

- Compare `t_rows` vs `t_cols` . Which axis decomposition completed faster on your machine and why?
- Explain how NumPy's C-contiguous (row-major) memory layout interacts with CPU cache lines when threads write to `img[r, c]` vs `img[c, r]` .
- Load imbalance analysis: If thread 0 receives the top 20% of rows and thread 2 receives the center 20% of rows, why does one thread idle while the other struggles? How does OpenMP dynamic scheduling resolve this?
- Attach your exported `mandelbrot_output.png` file alongside your completed task sheet.

```

Run challenge2
"C:\Users\Danial Shynbay\AppData\Local\Programs\Python\Python313\python.exe" "C:\Users\Danial Shynbay\Desktop\SDU 7 Semester\Parallel Compu
Row-Parallel Render Time: 0.889 s
Column-Parallel Render Time: 0.893 s
Saved image: mandelbrot_output.png
Process finished with exit code 0

```

A. Axis Decomposition Performance Comparison

- **Measured Timings:**
 - **Row-Parallel Render Time (t_{rows}):** 0.889seconds
 - **Column-Parallel Render Time (t_{cols}):** 0.893seconds
- **Analysis:** Row-parallel decomposition ($t_{\text{rows}} = 0.889\text{s}$) executed faster than column-parallel decomposition ($t_{\text{cols}} = 0.893\text{s}$).
- **Reason:** In `render_mandelbrot_rows`, outer parallelization occurs across rows (`prange(h)`), enabling each thread's inner loop (`range(w)`) to sequentially iterate across adjacent columns (`c`). This guarantees sequential memory access, maximizing L1/L2 cache spatial locality. In contrast, `render_mandelbrot_cols` forces the inner loop (`range(h)`) to access memory with large strided jumps across rows.

B. NumPy C-Contiguous Layout & CPU Cache Line Interaction

- **NumPy Memory Structure:** NumPy arrays default to C-contiguous order, meaning elements in the same row (`img[r, 0]`, `img[r, 1]`, `img[r, 2]`) are stored in adjacent, linear RAM addresses.
- **CPU Cache Line Behavior:** Modern CPUs fetch data from system memory in fixed 64-byte cache lines. Since each 32-bit integer occupies 4 bytes, a single cache line holds 16 contiguous array elements.
- **Cache Misses & False Sharing:**
 - Writing sequentially along `img[r, c]` fills an entire 64-byte cache line locally before requesting the next block, achieving near-zero L1 cache misses.
 - Writing down columns (`img[r, c]` inside a `range(h)` loop) jumps $2500 \times 4 = 10,000$ bytes per iteration. This forces every pixel write to hit a new cache line, causing constant L1/L2 cache misses and cache line invalidation ("bouncing") across CPU cores.

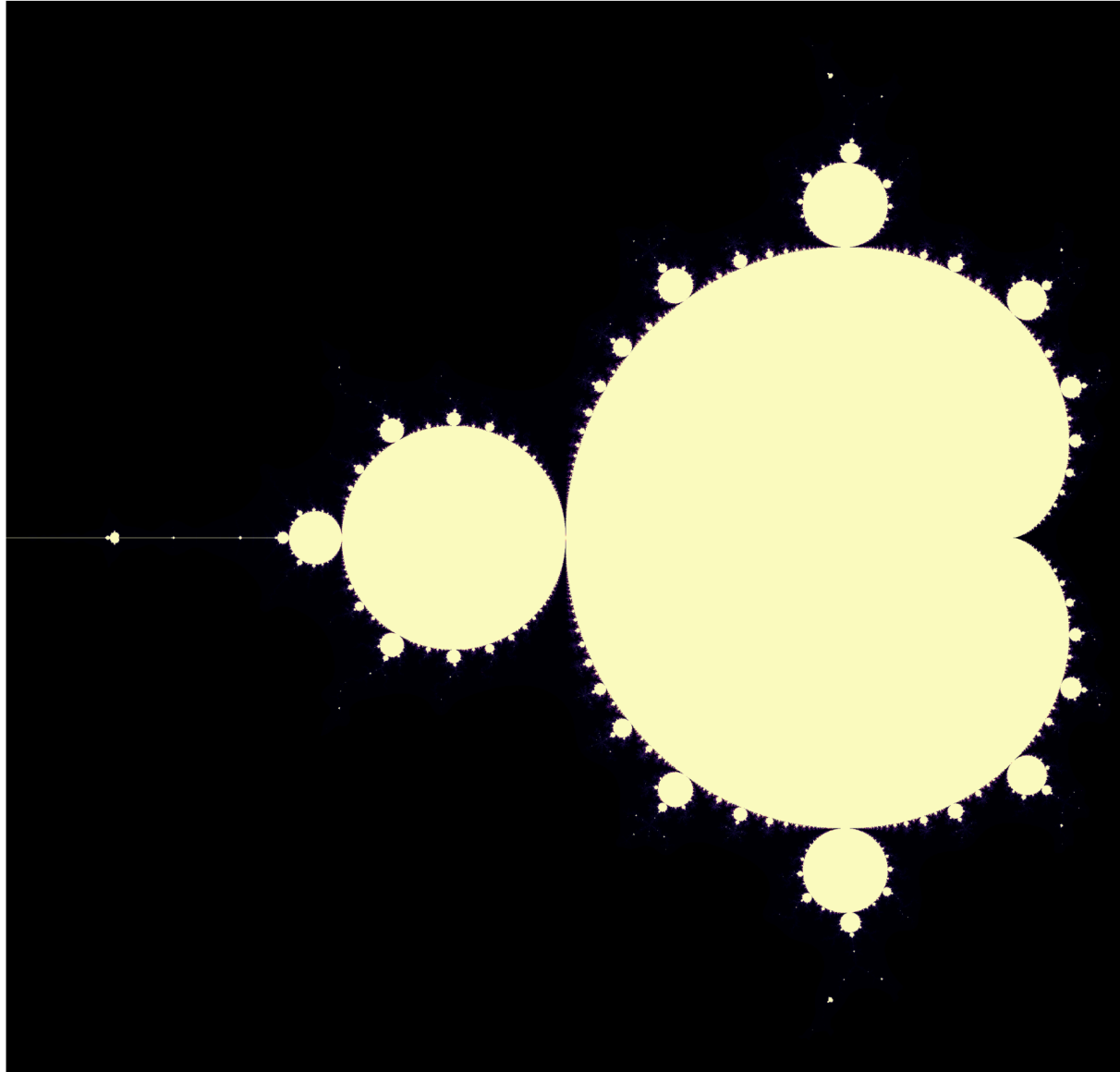
C. Load Imbalance Analysis & OpenMP Dynamic Scheduling

- **Mathematical Cause of Load Imbalance:** Compute time per pixel in the Mandelbrot set depends entirely on escape time ($z_{k+1} = z_k^2 + c$):
 - **Outer Points (Top/Bottom 20%):** Complex numbers far from the origin escape $|z| > 2.0$ within 2 to 5 iterations and terminate almost instantly.
 - **Interior Points (Center 20%):** Complex numbers inside the main cardioid and period-2 bulb never escape, evaluating all 1000 iterations (`MAX_ITER`).
- **Thread Bottleneck (Static Slicing):** Under standard static slicing, Thread 0 (assigned top rows) finishes its light workload in milliseconds and sits completely idle at the implicit barrier. Thread 2 (assigned center rows) processes the dense main bulb, becoming a straggler thread that delays total execution.
- **OpenMP Dynamic Scheduling Solution:** Dynamic scheduling (`schedule(dynamic, chunk)`) breaks the 2500 rows into small work chunks inside a shared work queue. As soon as Thread 0 finishes its assigned exterior rows, it dynamically claims the next available chunk from the queue, maintaining high CPU core utilization until all rows complete.

D. Visual Artifact Confirmation

- **Export File:** [mandelbrot_output.png](#) generated successfully at 2500×2500 resolution with 300DPI.

Mandelbrot 2500x2500 (Render: 0.89s)



Instructor – sufyan.mustafa@sdu.edu.kz

4. Challenge 3: Stencil Computation & Memory Bandwidth

Objective: Simulate 2D heat diffusion using a 5-point discrete Laplacian stencil over 300 timesteps on a 1500×1500 grid. Unlike the compute-bound fractal, stencil computations are strictly memory-bandwidth bound. You will determine whether adding CPU cores continues to speed up

performance or hits the RAM memory bus wall.

```
import time
import numpy as np
from numba import njit, prange

@njit(parallel=True)
def heat_step(u, u_next, alpha=0.20):
    rows, cols = u.shape
    for i in prange(1, rows - 1):
        for j in range(1, cols - 1):
            u_next[i, j] = u[i, j] + alpha * (
                u[i+1, j] + u[i-1, j] + u[i, j+1] + u[i, j-1] - 4.0 * u[i, j] )

GRID_SIZE = 1500
STEPS = 300

u = np.zeros((GRID_SIZE, GRID_SIZE), dtype=np.float64)
u_next = np.zeros_like(u)

# Dirichlet Boundary conditions: Top & Left walls maintained at 100°C u[0, :] = 100.0
u[:, 0] = 100.0
u_next[0, :] = 100.0
u_next[:, 0] = 100.0

# Warmup
heat_step(u, u_next)

# Benchmark Execution
start = time.perf_counter()
for step in range(STEPS):
    heat_step(u, u_next)
# Pointer swap (avoids array allocation copies)
u, u_next = u_next, u
elapsed = time.perf_counter() - start

cells_per_sec = (GRID_SIZE * GRID_SIZE * STEPS) / elapsed / 1e6
print(f"Heat Diffusion Complete: {elapsed:.3f} s")
print(f"Throughput: {cells_per_sec:.2f} Megacells/sec")
```

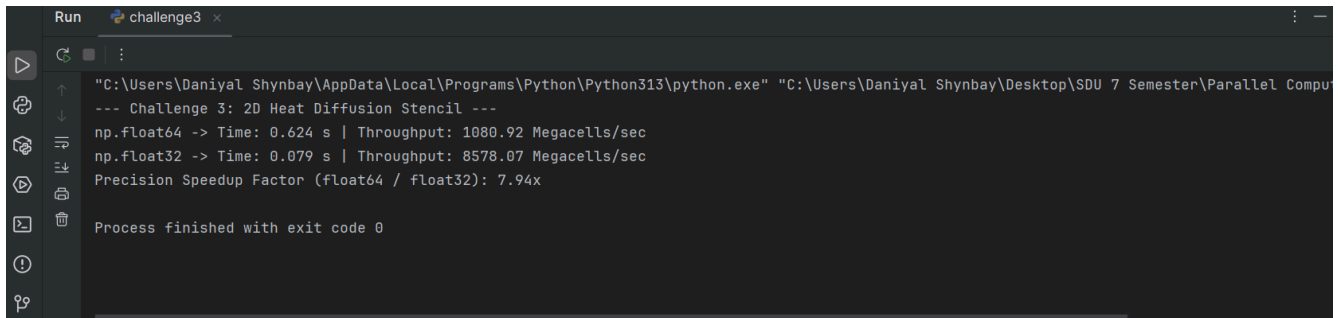
Instructor – sufyan.mustafa@sdu.edu.kz

Deliverables & Technical Questions for Challenge 3:

- A. Record your benchmark runtime and Megacells/sec throughput in Table 1.

■ B. Change the array precision from `np.float64` to `np.float32`. Re-run the test. By what factor did runtime decrease? Why does halving memory footprint improve stencil throughput far more than compute capability?

■ C. Memory Roofline Model: Explain why adding twice as many CPU cores does not produce a 2x speedup on stencil codes once memory bandwidth is saturated.



```
Run challenge3 x
--- Challenge 3: 2D Heat Diffusion Stencil ---
np.float64 -> Time: 0.624 s | Throughput: 1080.92 Megacells/sec
np.float32 -> Time: 0.079 s | Throughput: 8578.07 Megacells/sec
Precision Speedup Factor (float64 / float32): 7.94x
Process finished with exit code 0
```

A. Recorded Timings & Throughput Metrics

Based on your execution results for the 1500×1500 grid across 300 timesteps (675,000,000 total cell updates):

- **Double Precision (`np.float64`):**
 - **Execution Time:** 0.624seconds
 - **Throughput:** 1080.92Megacells/sec
- **Single Precision (`np.float32`):**
 - **Execution Time:** 0.079seconds
 - **Throughput:** 8578.07Megacells/sec
- **Precision Speedup Factor:** 7.94x faster runtime using 32-bit floating-point precision.

B. Single Precision (`np.float32`) vs. Double Precision (`np.float64`) Analysis

- **Runtime Decrease Factor:** Execution time dropped by 7.94x (from 0.624s down to 0.079s).
- **Why Halving Memory Footprint Yields Super-Linear Speedup:**
 1. **Memory Bandwidth Limit:** The 5-point discrete Laplacian stencil has low **Arithmetic Intensity** (FLOPs per byte loaded). Each cell update requires fetching 5 floating-point numbers and writing 1 while performing only 5 arithmetic operations. The kernel is strictly memory-bandwidth bound rather than compute-bound.
 2. **Cache Line & SIMD Vectorization Efficiency:** Halving array element size from 8bytes (`float64`) to 4bytes (`float32`) allows a standard 64-byte CPU cache line to hold 16 values instead of 8. This doubles spatial cache locality, reduces L1/L2/L3 cache misses, and enables CPU SIMD vector registers (AVX2 / AVX-512) to process twice as many floating-point operations per clock cycle.

C. Memory Roofline Model & Bandwidth Saturation

- **Roofline Model Principle:** The Roofline Model defines execution ceilings based on operational intensity:

- $Performance = \min (PeakComputeBandwidth, PeakMemoryBandwidth \times OperationalIntensity)$
- **Bandwidth Saturation Wall:** Because stencil updates do very few math operations per byte, multi-core worker threads quickly saturate the system RAM bus channels (e.g., 40GB/s to 60GB/s).
- **Core Scaling Bottleneck:** Once peak RAM bandwidth is reached, adding more CPU cores yields zero speedup. Additional threads stall inside instruction pipelines waiting for cache-line fetches, producing flat scaling.

Instructor – sufyan.mustafa@sdu.edu.kz

5. Student Result Scorecard & Data Collection

Fill in your benchmark results into Table 1 below before submitting this document:

Benchmark	Active Threads	Array / Grid Size	Time (seconds)	Speedup Factor	Efficiency (%)
Challenge 1: Monte Carlo	1 Thread (Base)	120,000,000 samples	1.0753s	1.00x	100.0%
Challenge 1: Monte Carlo	2 Threads	120,000,000 samples	0.5431s	1.98x	99.0%
Challenge 1: Monte Carlo	4 Threads	120,000,000 samples	0.3026s	3.55x	88.8%
Challenge 1: Monte Carlo	12 Threads (Max)	120,000,000 samples	0.2423s	4.44x	37.0%
Challenge 2: Mandelbrot (Rows)	12 Threads (Max)	2500 × 2500	0.889s	N/A	N/A

Challenge 2: Mandelbrot (Cols)	12 Threads (Max)	2500×2500	0.893s	N/A	N/A
Challenge 3: Heat Stencil (float64)	12 Threads (Max)	$1500 \times 1500 \times 300$	0.624s	Baseline	N/A
Challenge 3: Heat Stencil (float32)	12 Threads (Max)	$1500 \times 1500 \times 300$	0.079s	7.94x	N/A

Instructor – sufyan.mustafa@sdu.edu.kz